Unidad 9

```
1. Programación Síncrona vs. Asíncrona
Síncrona: Las tareas se realizan paso a paso, en orden. Si
una tarea tarda mucho (como una respuesta de servidor), blo
quea el programa y la aplicación parece congelada.
+1

Asíncrona: Permite realizar tareas simultáneamente sin espe
rar a que una termine para pasar a la siguiente. Esto hace
que el programa sea capaz de hacer más cosas en menos tiemp
o y no bloquea la ejecución del código.
+1

Ejemplo básico con setTimeout:
El método setTimeout ejecuta una función después de un tiem
po específico sin detener el resto del código.

JavaScript

console.log("Inicio del guión");

setTimeout(function() {
    console.log("Primer tiempo muerto completado");
}, 2000); // Se ejecuta tras 2 segundos

console.log("Fin del guión");
// La consola mostrará: Inicio -> Fin -> Primer tiempo muer
to
2. Callbacks (Retrollamadas)
Una callback es una función que se pasa como argumento a ot
ra función y se ejecuta después de que se haya completado l
```

```
a primera. Se usan mucho para manejar eventos o recibir dat
os de un servidor.
+1


Problema (Callback Hell): Si encadenamos muchas callbacks u
na dentro de otra, se crea una estructura piramidal difícil
de leer y mantener.
+1

Ejemplo de Callback:

JavaScript

function tareaAsincrona(mensaje, callback) {
    setTimeout(function() {
        console.log(mensaje);
        callback(); // Se ejecuta al terminar la espera
    }, 2000);
}

console.log('Inicio');
tareaAsincrona('Procesando datos...', function() {
    console.log('Tarea terminada');
});
console.log('Fin inmediato');
3. Promesas (Promises)
Para solucionar el Callback Hell, usamos Promesas. Una prom
esa es un objeto que representa un valor o acción futura.

Estados:


Pendiente (pending): Estado inicial.


Cumplida (fulfilled): Éxito (se llama a resolve).
```

```
Rechazada (rejected): Fallo (se llama a reject).


Manejo: Se usa .then() para manejar el éxito y .catch() par
a los errores.

Promise.all: Ejecuta varias promesas en paralelo y espera a
que todas se cumplan. Si una falla, toda la operación fall
a.
+1

Ejemplo de Promesa:

JavaScript

function simularConsultaServidor() {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            const numero = Math.random();
            if (numero >= 0.5) {
                resolve('Datos recibidos correctamente');
            } else {
                reject('Error en el servidor');
            }
        }, 2000);
    });
}

simularConsultaServidor()
    .then((mensajeOk) => {
        console.log('Éxito:', mensajeOk);
    })
    .catch((mensajeError) => {
        console.error('Fallo:', mensajeError);
    });
4. Funciones Asíncronas (Async / Await)
Es una sintaxis más moderna y legible para trabajar con pro
```

```
mesas, haciendo que el código asíncrono parezca síncrono.


async: Declara la función.


await: Pausa la ejecución dentro de la función hasta que la
promesa se resuelve.


Manejo de errores: Se utiliza el bloque try / catch.

Ejemplo Async/Await:

JavaScript

function esperar(tiempo) {
    return new Promise(resolve => setTimeout(resolve, tiemp
o));
}

async function procesoCompleto() {
    try {
        console.log('Paso 1: Iniciando...');
        await esperar(2000); // Espera 2 segundos
        console.log('Paso 2: Datos recibidos');
    } catch (error) {
        console.error('Error:', error);
    }
}

procesoCompleto();
5. AJAX y Fetch API

AJAX permite actualizar contenido de una web dinámicamente
sin recargar la página completa. Aunque originalmente usaba
XML, ahora el estándar es JSON.
+1
```

```
XMLHttpRequest: La forma antigua de hacer peticiones. Es má
s compleja y verbosa.

Fetch API: La forma moderna, basada en Promesas. Es más sen
cilla y eficiente.

Comparativa: Cargar un JSON


Opción A: Fetch (Recomendada)

JavaScript

fetch('datos.json')
    .then(response => response.json()) // Convierte respues
ta a JSON
    .then(datos => {
        console.log('Nombre:', datos.nombre);
        console.log('Email:', datos.email);
    })
    .catch(error => console.log(error));

Opción B: XMLHttpRequest (Antigua)

JavaScript

var xhr = new XMLHttpRequest();
xhr.open('GET', 'datos.json', true);
xhr.onreadystatechange = function() {
    if (xhr.readyState == 4 && xhr.status == 200) {
        var datos = JSON.parse(xhr.responseText);
        console.log('Nombre:', datos.nombre);
    }
};
xhr.send();
6. FormData
Para enviar datos de formularios (especialmente útil para a
```

```
rchivos o grandes volúmenes de datos), se utiliza el objeto
FormData. Permite añadir pares clave-valor y se envía habit
ualmente mediante POST.
+1

JavaScript

const form = document.getElementById("miFormulario");

form.addEventListener("submit", async (e) => {
    e.preventDefault();
    // Crea el objeto con los datos del form automáticament
e
    const formData = new FormData(form);

    // Se puede enviar directamente en el body del fetch
    try {
        const respuesta = await fetch("procesar.php", {
            method: "POST",
            body: formData
        });
        const texto = await respuesta.text();
        console.log(texto);
    } catch (error) {
        console.error(error);
    }
});
```